Experiment 7

Random Forest

Aim

To implement the Random Forest algorithm as an ensemble of multiple decision trees using random sampling and feature selection, and to evaluate its performance for classification tasks.

Theory

A Random Forest is a supervised ensemble learning algorithm used for both classification and regression tasks. Instead of relying on a single decision tree (which can easily overfit the training data), Random Forest builds many decision trees during training and combines their outputs to produce a more accurate and stable prediction. Each individual tree in the forest may be a relatively weak learner on its own, but when many such trees vote together, the ensemble becomes a strong learner with significantly improved generalisation.

1. From a Single Decision Tree to a Forest

A single Decision Tree learns by recursively splitting data on features to create decision rules. While simple and interpretable, it has a key limitation: high variance. Small changes in training data can produce a completely different tree, leading to overfitting. Random Forest addresses this by building an ensemble of many decision trees and aggregating their predictions. The core idea is:

Many weak learners, each slightly different, can combine to form one strong learner.

The two sources of randomness that make the trees diverse are Bootstrap Sampling (Bagging) and Random Feature Selection, explained below.

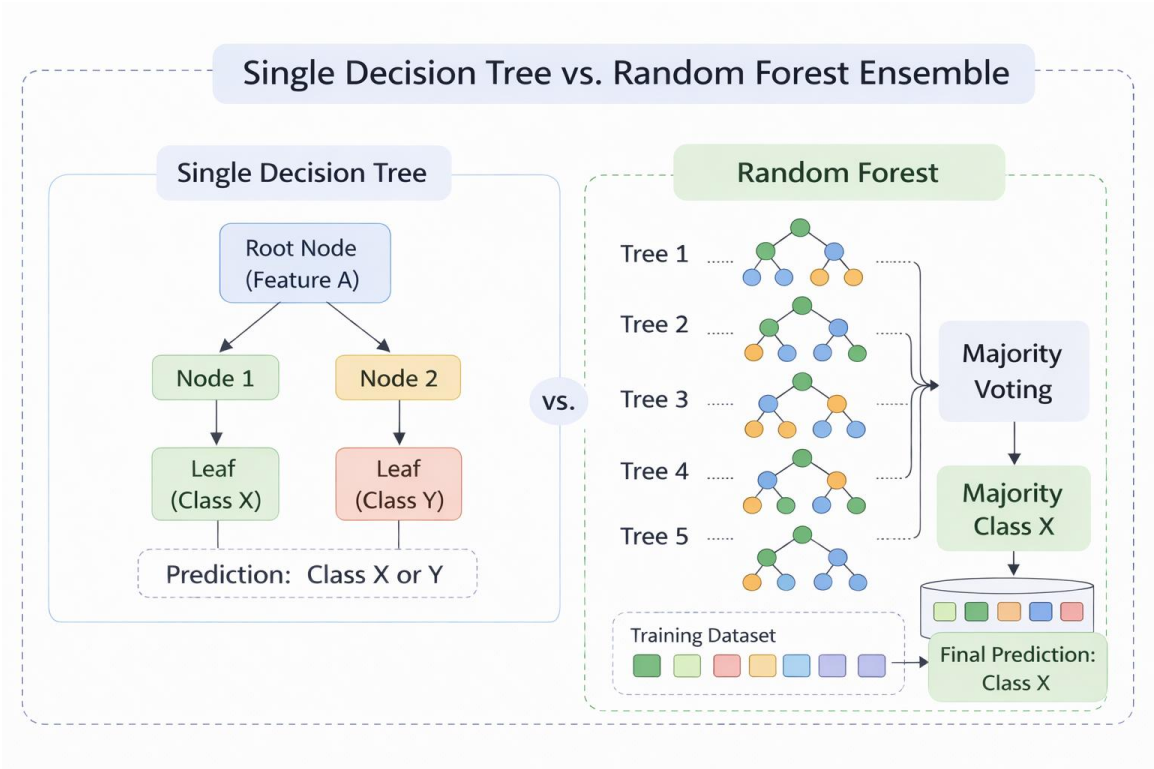

Figure 1 - A single Decision Tree (left) vs. a Random Forest ensemble (right). Each tree votes independently and the final prediction is made via majority voting.

2. Bootstrap Sampling (Bagging)

Bagging (Bootstrap Aggregating) is the first source of randomness. For each tree in the forest:

1. A bootstrap sample is created by randomly selecting N samples with replacement from the original training set of N samples.
2. On average, each bootstrap sample contains about 63% of unique training samples (since sampling is with replacement, some samples appear more than once and some are left out).
3. The remaining ~37% of samples that are not selected for a given tree are called Out-of-Bag (OOB) samples. These act as a built-in validation set for that tree.

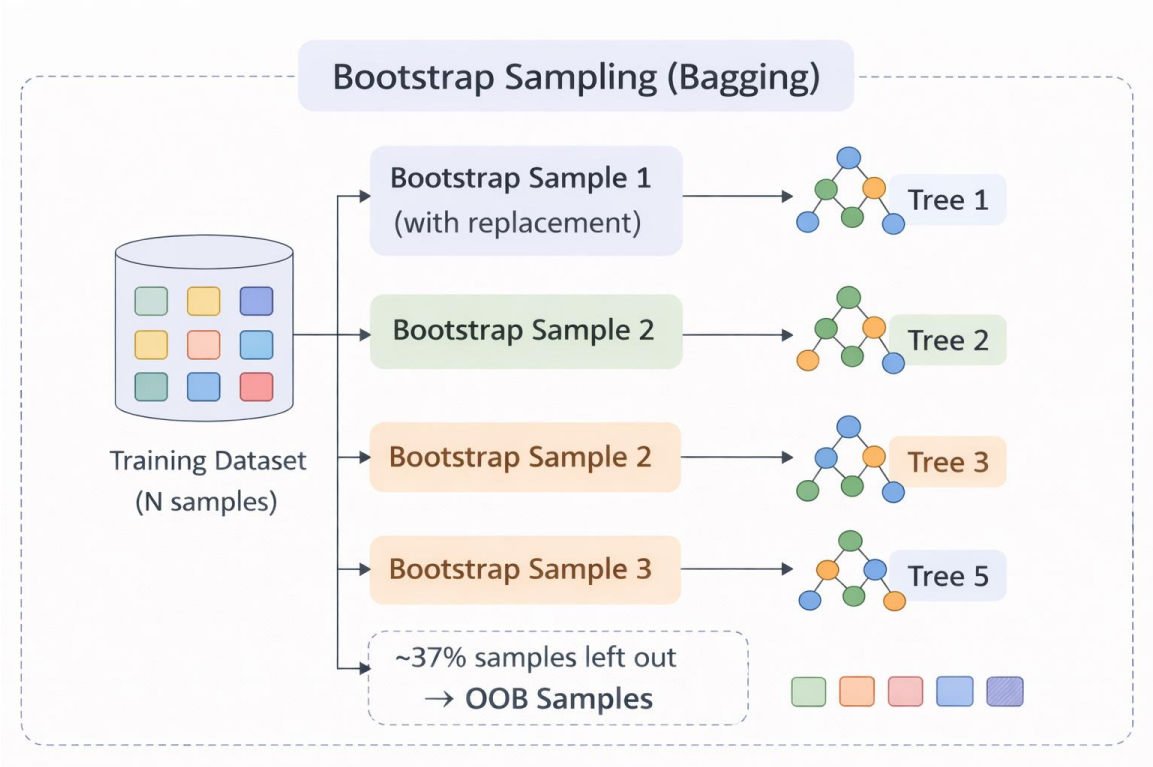

Figure 2 - Bootstrap sampling: each tree receives a different random subset (with replacement) of the training data. Unselected samples become OOB samples.

Because each tree is trained on a different subset, the trees learn slightly different patterns, making their errors less correlated. When combined, the variance of the overall model drops significantly.

3. Random Feature Selection

The second source of randomness occurs at every split during tree construction. Instead of evaluating all available features to find the best split, each node only considers a random subset of m features (typically $m = \sqrt{p}$ for classification, where p is the total number of features).

This prevents dominant features from appearing at the root of every tree, forcing the forest to explore a wider variety of feature interactions. Without this step, all trees would look very similar despite using different bootstrap samples.

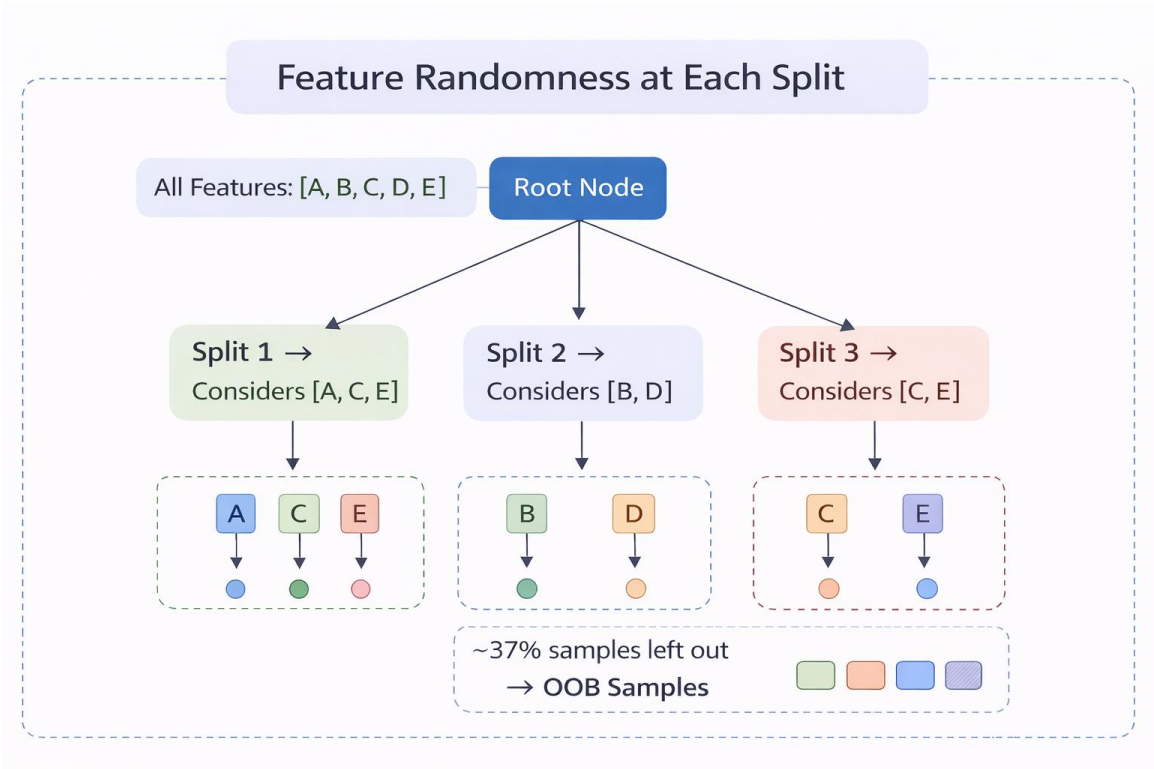

Figure 3 - At each node, only a random subset of features (highlighted) is evaluated. Different nodes and different trees use different subsets.

4. Bagging + Feature Randomness: Working Together

The power of Random Forest comes from combining both sources of randomness:

Source of Randomness	What it does	Effect
Bootstrap Sampling	Each tree trains on a different data subset	Reduces variance by decorrelating trees
Feature Randomness	Each split considers a random feature subset	Prevents dominant features from making trees similar
Combined Effect	Trees are diverse in both data and features	Ensemble is robust, low-variance, and generalises well

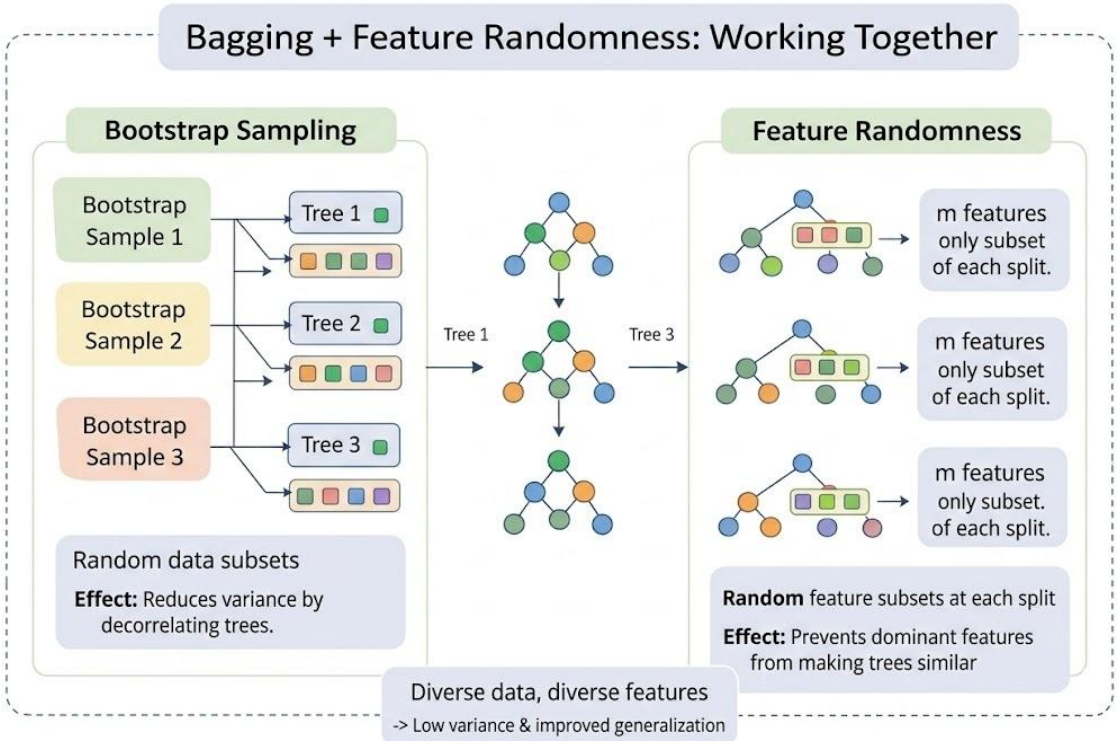

Figure 4 - Random Forest = Bagging (different data subsets) + Feature Randomness (different feature subsets at each split). This dual randomness produces a diverse, high-performing ensemble.

5. Splitting Criteria

At each node, the best split is chosen from the randomly selected feature subset using an impurity measure. The two commonly used criteria are:

Gini Impurity measures the probability of misclassifying a randomly chosen sample:

$$Gini = 1 - \sum_{i=1}^{C} p_i^2$$

where p_i is the proportion of samples belonging to class i, and C is the total number of classes. A Gini value of 0 indicates a perfectly pure node (all samples belong to one class). Higher values indicate more impurity.

Entropy measures the amount of uncertainty or disorder in a node:

$$Entropy = -\sum_{i=1}^{C} p_i \, log_2(p_i)$$

Entropy is 0 for a pure node and reaches its maximum when classes are equally distributed. Information Gain, which is the reduction in entropy after a split, is used to select the best feature and threshold.

Both criteria generally produce similar trees. Gini is computationally slightly faster (no logarithm), while Entropy tends to produce more balanced splits.

6. Prediction Mechanism

Once all trees are trained, a new sample is passed through every tree in the forest:

- Classification: Each tree votes for a class. The class with the majority of votes becomes the final prediction.
- Regression: Each tree outputs a numeric value. The average of all outputs is the final prediction.

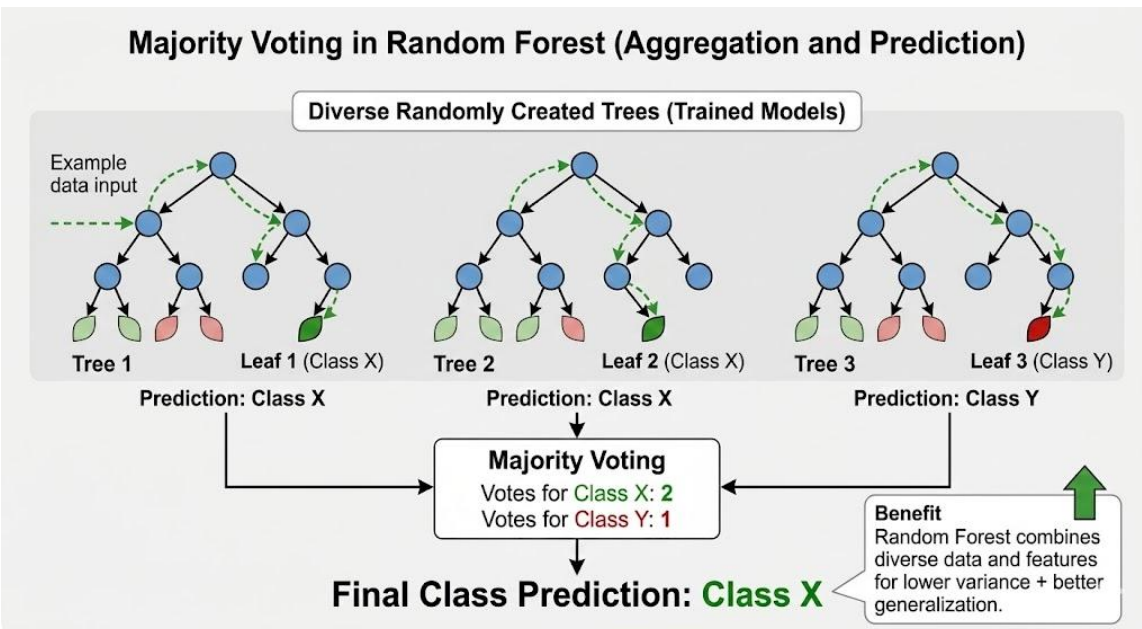

Figure 5 - Prediction via majority voting: each tree votes independently, and the class with the most votes wins.

Example

Suppose a Random Forest contains 5 trees and predicts whether an email is Spam or Not Spam:

Tree	Prediction
Tree 1	Spam
Tree 2	Not Spam
Tree 3	Spam
Tree 4	Spam
Tree 5	Not Spam

Majority vote = Spam (3 out of 5 trees). Even though two trees predicted "Not Spam", the ensemble correctly classifies the email by following the majority.

7. Out-of-Bag (OOB) Error Estimation

A unique advantage of Random Forest is built-in cross-validation through OOB samples. Since each tree was trained on only ~63% of the data, the remaining ~37% (OOB samples) can be used to evaluate that tree's performance without needing a separate validation set.

The OOB error is computed by:

4. For each training sample, collecting predictions only from trees where it was not in the bootstrap sample.
5. Using majority voting on those predictions to classify the sample.
6. Computing the overall misclassification rate.

The OOB error is an unbiased estimate of the generalisation error and closely approximates leave-one-out cross-validation.

8. Key Hyperparameters

Parameter	Description	Typical Values
n_estimators	Number of trees in the forest	100–500 (more trees → better accuracy, diminishing returns)
max_features	Number of features considered at each split	√p (classification), p/3 (regression)
max_depth	Maximum depth of each tree	None (grow fully) or limited to prevent overfitting
min_samples_split	Minimum samples required to split a node	2–10
min_samples_leaf	Minimum samples required at a leaf node	1–5

Increasing n_estimators generally improves performance but increases computation time. Reducing max_features increases tree diversity but may reduce individual tree accuracy. The balance between these parameters is key to a well-performing Random Forest.

9. Algorithm

- Step 1: Set parameters:
  n = number of trees to build
  m = number of features to consider at each split (usually √total_features for classification)

- Step 2: For each tree i = 1 to n:
  - Step 2a: Bootstrap Sampling

Randomly select N samples with replacement from training data
This creates a different dataset for each tree
About 37% of original data is left out (Out-of-Bag samples)

- Step 2b: Build Decision Tree with Random Feature Selection
  At each node:
  Randomly select m features from all features
  Find the best split using ONLY these m features
  Split the node
  Grow tree fully (no pruning)

- Step 3: Store all n trees

- Step 4: For prediction (Classification):
  Pass new sample through all trees
  Each tree gives one vote
  Final prediction = class with majority votes

- Step 5: For prediction (Regression):
  Pass new sample through all trees
  Each tree gives one numeric prediction
  Final prediction = average of all tree predictions

- Step 6: Evaluate using OOB Error:
  For each sample, aggregate predictions from trees that did not include it
  Compute misclassification rate as OOB error estimate

- Step 7: Calculate Feature Importance:
  For each feature, sum up how much it reduced impurity across all trees

10. Decision Tree vs. Random Forest

Aspect	**Single Decision Tree**	**Random Forest**
Model Type	Single model	Ensemble of many trees
Variance	High (sensitive to data changes)	Low (averaging reduces variance)
Overfitting	Prone to overfitting	Resistant due to bagging + randomness
Accuracy	Moderate	Generally higher
Interpretability	Easy (one tree to follow)	Harder (many trees)
Training Time	Fast	Slower (many trees)
Feature Importance	Available but unstable	Stable and reliable

11. Merits of Random Forest

- High Accuracy & Robustness: Combining multiple trees reduces overfitting and usually gives better accuracy than a single decision tree.
- Handles Large & Complex Data: Works well with large datasets and can handle both numerical and categorical features.
- Feature Importance Estimation: Measures the importance of each feature based on impurity reduction, aiding feature selection.
- Built-in Validation: OOB error provides a reliable generalisation estimate without needing a separate validation set.
- Minimal Preprocessing: Does not require feature scaling or normalisation.

12. Demerits of Random Forest

- Computationally Expensive: Training many trees requires more time and memory compared to simpler models.
- Less Interpretability: The ensemble nature makes it harder to interpret compared to a single decision tree.
- Not Ideal for Real-Time Prediction: Multiple trees increase prediction latency.
- Large Model Size: Storing hundreds of trees consumes more memory than a single model.

Procedure

Part 1: Binary Classification (Bank Marketing Dataset)

The objective of this part is to classify bank customers into two categories (subscribed / not subscribed to a term deposit) using a Random Forest classifier, compare its performance against a single Decision Tree, and explore how key hyperparameters affect accuracy and generalisation.

Step 1: Import the required libraries: numpy, pandas, matplotlib, seaborn, and the relevant modules from sklearn (tree, ensemble, model_selection, metrics).

Step 2: Load the Bank Marketing dataset (bank_marketing.csv). The dataset contains approximately 45,211 records from direct phone-based marketing campaigns by a Portuguese bank. It includes 17 feature columns describing customer demographics, financial status, and campaign details, along with a binary target variable indicating subscription outcome (yes/no).

Step 3: Perform Exploratory Data Analysis:

- Use head(), info(), describe(), and isnull().sum() to understand the structure and quality of the data.
- Check the class distribution of the target variable using value_counts() to identify any class imbalance.

Step 4: Define Features and Target:

- Features (X): age, job, marital, education, default, housing, loan, contact, month, day_of_week, duration, campaign, pdays, previous, poutcome, emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m, nr.employed.
- Target (Y): subscription outcome (yes / no).
- Encode categorical features using LabelEncoder or pd.get_dummies().

Step 5: Split the dataset into 80% Training Set and 20% Testing Set using train_test_split() with a fixed random_state for reproducibility.

Step 6: Train a Single Decision Tree as Baseline:

- Initialize a DecisionTreeClassifier with max_depth=5 and criterion='gini'.
- Fit on the training data and record its test accuracy. This serves as a baseline to compare against the Random Forest.

Step 7: Train a Random Forest Classifier:

- Initialize RandomForestClassifier with n_estimators=100, criterion='gini', and random_state=42.
- Fit on the training data and record test accuracy.
- Compare the Random Forest accuracy with the single Decision Tree from Step 6 to observe the ensemble improvement.

Step 8: Hyperparameter Exploration:

- Vary n_estimators (e.g., 10, 50, 100, 200, 500): Train a Random Forest for each value, record the test accuracy, and plot accuracy vs. number of trees to observe diminishing returns.
- Vary max_depth (e.g., 3, 5, 10, 20, None): Observe the trade-off between underfitting and overfitting.
- Vary max_features (e.g., 'sqrt', 'log2', 0.5, 1.0): Observe how feature randomness affects performance and tree diversity.

Step 9: OOB Score Analysis:

- Train a RandomForestClassifier with oob_score=True and access oob_score_ after fitting.
- Compare the OOB score with the test set accuracy to verify that OOB provides a reliable estimate of generalisation without a separate validation set.

Step 10: Feature Importance Visualisation:

- Extract feature_importances_ from the trained Random Forest.
- Plot a horizontal bar chart to show the relative importance of each feature.
- Identify which features contribute most to the classification and discuss how this aids interpretation.

Step 11: Evaluate Model Performance:

- Generate predictions on the test set using the best Random Forest configuration.
- Compute Accuracy, Precision, Recall, and F1-Score.
- Plot a Confusion Matrix heatmap to visualise true positives, true negatives, false positives, and false negatives.
- Display the full Classification Report using classification_report().

Part 2: Multi-Class Classification (Seeds Dataset)

The objective of this part is to classify wheat seeds into three varieties (Kama, Rosa, and Canadian) based on geometric measurements, using a Random Forest classifier. This part reinforces ensemble concepts in a multi-class setting and includes comparative analysis and hyperparameter tuning.

Step 1: Import the required libraries: numpy, pandas, matplotlib, seaborn, and the relevant modules from sklearn.

Step 2: Load the Seeds dataset (Seeds_Dataset.csv). The dataset contains 210 wheat seed samples with 7 numerical features (Area, Perimeter, Compactness, Length of Kernel, Width of Kernel, Asymmetry Coefficient, Length of Kernel Groove) and a class label identifying the seed variety (1, 2, or 3).

Step 3: Perform Exploratory Data Analysis:

- Use head(), info(), describe(), and isnull().sum() to inspect the data.
- Check class distribution using value_counts().
- Plot pairwise scatter plots or histograms to visualise feature distributions across the three classes.

Step 4: Define Features and Target:

- Features (X): Area, Perimeter, Compactness, Length of Kernel, Width of Kernel, Asymmetry Coefficient, Length of Kernel Groove.
- Target (Y): Class (1, 2, or 3).

Step 5: Split the dataset into 80% Training Set and 20% Testing Set using train_test_split() with stratified sampling to preserve class proportions.

Step 6: Train a Single Decision Tree as Baseline:

- Initialize a DecisionTreeClassifier with criterion='gini'.
- Fit on the training data and record test accuracy for comparison.

Step 7: Train a Random Forest Classifier:

- Initialize RandomForestClassifier with n_estimators=150, criterion='gini', and random_state=42.
- Fit on the training data, record test accuracy, and compare with the Decision Tree baseline.

Step 8: Hyperparameter Exploration:

- Vary n_estimators (e.g., 10, 50, 100, 150, 300): Plot accuracy vs. number of trees.
- Vary max_depth (e.g., 3, 5, 10, None): Observe the impact on multi-class performance.
- Vary criterion ('gini' vs. 'entropy'): Compare the two splitting strategies and note any differences in accuracy or tree structure.

Step 9: OOB Score Analysis:

- Enable oob_score=True and compare oob_score_ with test accuracy.
- Discuss how OOB eliminates the need for cross-validation and provides a quick generalisation estimate.

Step 10: Feature Importance Visualisation:

- Extract and plot feature_importances_ as a bar chart.
- Identify which geometric measurements are most influential in distinguishing the three seed varieties.

Step 11: Evaluate Model Performance:

- Generate predictions on the test set.
- Compute Accuracy, Precision (macro), Recall (macro), and F1-Score (macro).
- Plot a Confusion Matrix heatmap to visualise per-class performance.
- Display the full Classification Report with per-class precision, recall, and F1-Score.